

WEEK -3 ASSIGNMENT

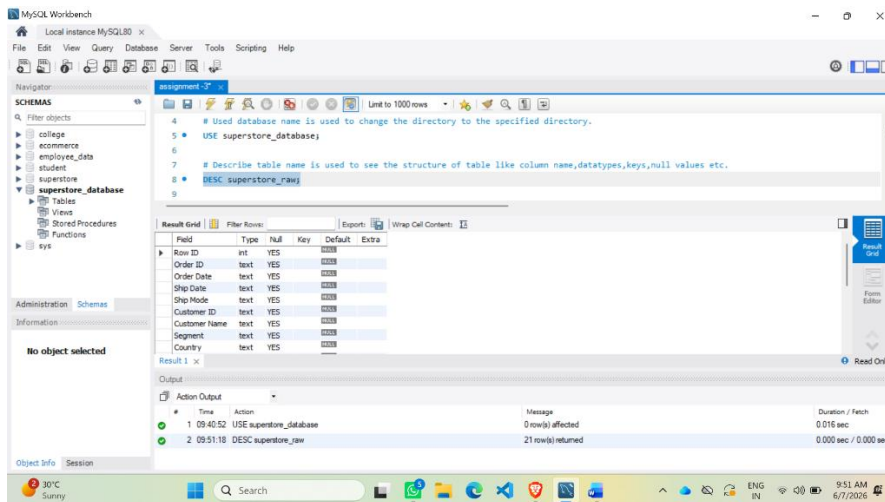
Objective:

Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

Tasks

Step 1: Setup Data

1. Import the Superstore dataset into a table named **superstore_raw**.
 - Created a new database named superstore_database.
 - In schema -> Right click on superstore_database -> Select table data import wizard -> Select file path -> Name the table as superstore_raw -> click on import



2. Create these 3 tables from it:

a. customers

CREATE TABLE table_name through this query we have created a table named customers with columns customer_id and customer_name.

```
CREATE TABLE customers(
customer_id VARCHAR(50),
customer_name VARCHAR(100)
);
```

b. orders

Through this query we have created a table named orders which contains order info like order_id, customer_id, product_id, order_date, sales.

```
CREATE TABLE orders (
order_id VARCHAR(30),
```

WEEK -3 ASSIGNMENT

```
customer_id VARCHAR(20),  
product_id VARCHAR(50),  
order_date DATE,  
sales DECIMAL(10,2)  
);
```

c. products

Through this query we have created a table named products which contains product info like product_id, product_name, category.

```
CREATE TABLE products (  
    product_id VARCHAR(50),  
    product_name VARCHAR(255),  
    category VARCHAR(100)  
);
```

3. Insert data into these tables using **SELECT DISTINCT**.

Through DISTINCT keyword we have inserted unique customer records from the raw dataset into the customers table.

```
INSERT INTO customers (customer_id, customer_name)
```

```
SELECT DISTINCT
```

```
    `Customer ID`,
```

```
    `Customer Name`
```

```
FROM superstore_raw;
```

Through DISTINCT keyword we have inserted unique product records from the raw dataset into the products table.

```
INSERT INTO products
```

```
SELECT DISTINCT
```

```
    `Product ID`,
```

```
    `Product Name`,
```

```
    Category
```

```
FROM superstore_raw;
```

WEEK -3 ASSIGNMENT

Through DISTINCT keyword we have inserted unique order records from the raw dataset into the orders table.

INSERT INTO orders

SELECT DISTINCT

`Order ID`,

`Customer ID`,

`Product ID`,

STR_TO_DATE(`Order Date`, '%m/%d/%Y'),

Sales

FROM superstore_raw;

Step 2: Perform Required Queries

Write and execute SQL queries for each of the following:

1. Find all orders where sales are greater than the average sales. (Subquery)

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
65  
66 # Subquery  
67 # Used to find orders whose sales value is greater than the average sales.  
68 *  
69 SELECT * FROM orders  
70 WHERE Sales > (SELECT AVG(Sales) FROM orders);
```

The Results grid displays the following data:

order_id	customer_id	product_id	order_date	sales
CA-2016-152156	CG-12520	FLR-CH-1000454	2016-11-08	261.96
CA-2016-152156	CG-12520	FLR-CH-1000454	2016-11-08	731.94
US-2015-109866	SO-20335	FLR-TA-1000577	2015-10-11	957.58
CA-2014-115812	BH-11730	TEC-PH-1000275	2014-06-09	907.15
CA-2014-115812	BH-11730	FLR-TA-1000139	2014-06-09	1706.18
CA-2014-115812	BH-11730	TEC-PH-1000203	2014-06-09	911.42
CA-2016-151259	BH-15076	OFF-ES-1001306	2016-12-05	407.58
CA-2014-108893	PC-28079	OFF-OF-1000486	2014-11-11	665.98
CA-2015-106320	EB-13870	FLR-TA-1000577	2015-09-25	1044.63

The Output pane shows the execution results:

Step	Action	Message	Duration / Fetch
2	DESC superstore_raw	21 rows returned	0.000 sec / 0.000 sec
3	INSERT INTO customers (customer_id, customer_name) SELECT DISTINCT	793 rows affected Records: 793 Duplicates: 0 Warnings: 0	0.031 sec
4	SELECT * FROM orders WHERE Sales > (SELECT AVG(Sales) FROM orders) LHM	1000 rows returned	0.000 sec / 0.000 sec

2. Find the highest sales order for each customer. (Subquery)

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
69  
70 WHERE Sales > (SELECT AVG(Sales) FROM orders);  
71  
72 # Used to find the highest sales order placed by each customer.  
73 *  
74 SELECT * FROM orders  
75 WHERE sales IN (SELECT MAX(sales) FROM orders GROUP BY customer_id)  
76  
77 # CTE  
78 # Used to calculate total sales generated by each customer using a CTE.  
79 WITH customer_sales AS  
80 {
```

The Results grid displays the following data:

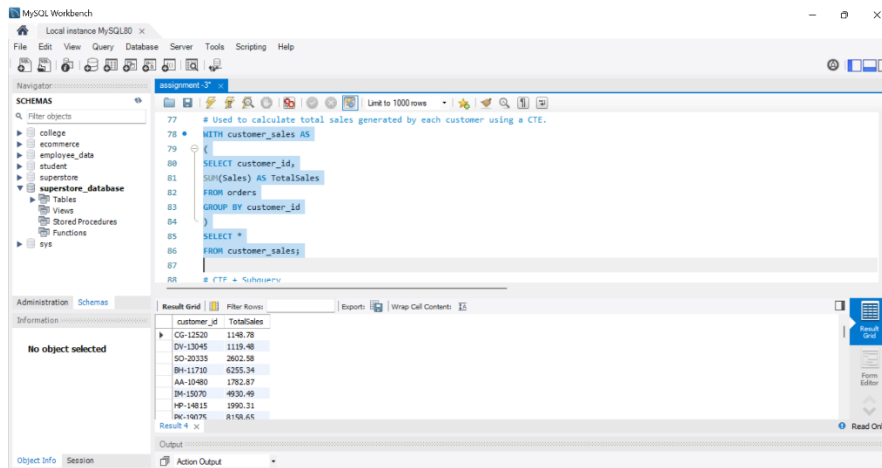
order_id	customer_id	product_id	order_date	sales
CA-2016-152156	CG-12520	FLR-CH-1000454	2016-11-08	731.94
US-2015-109866	SO-20335	FLR-TA-1000577	2015-10-11	957.58
CA-2014-115812	BH-11730	FLR-TA-1000139	2014-06-09	1706.18
CA-2015-106320	EB-13870	FLR-TA-1000577	2015-09-25	1044.63
US-2015-109839	TP-11020	FLR-BO-1000424	2015-09-17	3063.43
CA-2016-117990	GH-14485	TEC-PH-1000487	2016-12-08	1097.94
CA-2015-117415	SH-20730	FLR-BO-1000245	2015-12-27	532.40
CA-2016-108816	JH-15363	TEC-PH-1000247	2016-12-11	1028.95
CA-2016-115882	TP-21055	FLR-CH-1000368	2016-06-17	319.41

The Output pane shows the execution results:

Step	Action	Message	Duration / Fetch
2	DESC superstore_raw	21 rows returned	0.000 sec / 0.000 sec
3	INSERT INTO customers (customer_id, customer_name) SELECT DISTINCT	793 rows affected Records: 793 Duplicates: 0 Warnings: 0	0.031 sec
4	SELECT * FROM orders WHERE Sales > (SELECT AVG(Sales) FROM orders) LHM	1000 rows returned	0.000 sec / 0.000 sec

WEEK -3 ASSIGNMENT

3. Calculate total sales for each customer. (CTE)



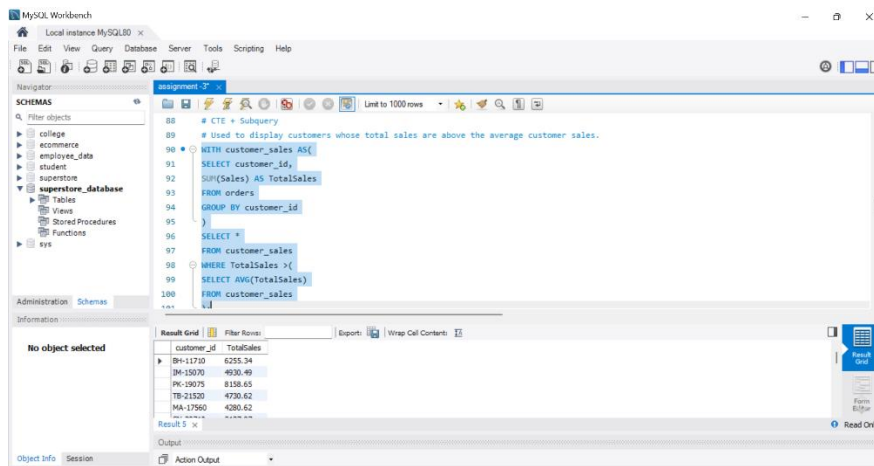
The screenshot shows the MySQL Workbench interface with a query editor containing a CTE query. The query is as follows:

```
77 # Used to calculate total sales generated by each customer using a CTE.
78 WITH customer_sales AS
79 (
80 SELECT customer_id,
81 SUM(Sales) AS TotalSales
82 FROM orders
83 GROUP BY customer_id
84 )
85 SELECT *
86 FROM customer_sales;
87
88 # CTE + Subquery
```

The result grid shows the following data:

customer_id	TotalSales
CG-12520	1148.78
DV-13045	1119.46
SO-20335	2602.58
BH-11730	6255.34
AA-19480	1782.87
JM-15070	4930.49
HP-14815	1990.31
PK-19075	8138.65

4. Find customers whose total sales are above average. (CTE + Subquery)



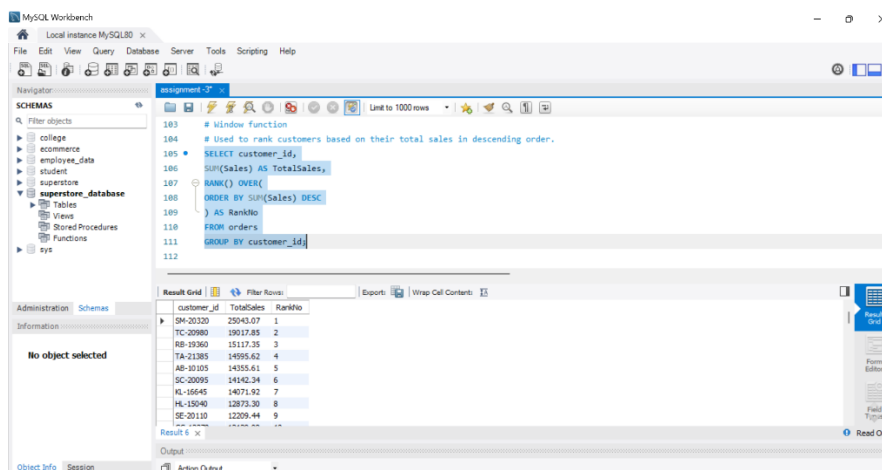
The screenshot shows the MySQL Workbench interface with a query editor containing a CTE query. The query is as follows:

```
88 # CTE + Subquery
89 # used to display customers whose total sales are above the average customer sales.
90 WITH customer_sales AS
91 (
92 SELECT customer_id,
93 SUM(Sales) AS TotalSales
94 FROM orders
95 GROUP BY customer_id
96 )
97 SELECT *
98 FROM customer_sales
99 WHERE TotalSales > (
100 SELECT AVG(TotalSales)
101 FROM customer_sales
102 )
```

The result grid shows the following data:

customer_id	TotalSales
BH-11730	6255.34
JM-15070	4930.49
PK-19075	8138.65
TB-21320	4730.62
JA-17960	4280.62

5. Rank all customers based on total sales. (Window Function)



The screenshot shows the MySQL Workbench interface with a query editor containing a window function query. The query is as follows:

```
103 # Window Function
104 # Used to rank customers based on their total sales in descending order.
105 SELECT customer_id,
106 SUM(Sales) AS TotalSales,
107 RANK() OVER(
108 ORDER BY SUM(Sales) DESC
109 ) AS RankNo
110 FROM orders
111 GROUP BY customer_id;
112
```

The result grid shows the following data:

customer_id	TotalSales	RankNo
SM-20330	25043.07	1
TC-20980	19017.85	2
MB-09360	15117.35	3
TA-21385	14995.62	4
AB-10105	14355.61	5
SC-20095	14142.34	6
HL-06645	14071.92	7
HL-15040	12873.30	8
SE-20110	12209.44	9

WEEK -3 ASSIGNMENT

6. Assign row numbers to each order within a customer. (*Window Function + PARTITION BY*)

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
114 # Used to assign a row number to each order within every customer group.
115 SELECT customer_id,
116        order_id,
117        Sales,
118        ROW_NUMBER() OVER(
119        PARTITION BY customer_id
120        ORDER BY Sales DESC
121        ) AS RowNum
122 FROM orders;
123
```

The Results Grid shows the output of the query:

customer_id	order_id	Sales	RowNum
AA-10315	CA-2016-103982	3930.07	1
AA-10315	CA-2014-128055	673.57	2
AA-10315	CA-2016-103982	431.98	3
AA-10315	CA-2017-147039	362.94	4
AA-10315	CA-2014-128055	52.98	5
AA-10315	CA-2016-103982	41.72	6
AA-10315	CA-2015-121391	26.96	7
AA-10315	CA-2014-138100	14.94	8
AA-10315	CA-2014-138100	14.56	9

7. Display top 3 customers based on total sales. (*Window Function*)

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
123 # Window function
124 # Used to retrieve the top 3 customers based on total sales.
125 WITH customer_sales AS(
126 SELECT customer_id,
127        SUM(Sales) AS TotalSales,
128        RANK() OVER(
129        ORDER BY SUM(Sales) DESC
130        ) AS RankNo FROM orders
131 ) AS RankNo FROM customer_sales
132 GROUP BY customer_id
133 )
134 SELECT * FROM customer_sales
135 WHERE RankNo <= 3;
136
```

The Results Grid shows the output of the query:

customer_id	TotalSales	RankNo
SM-20320	25043.07	1
TC-20980	19017.85	2
RB-29360	15117.35	3

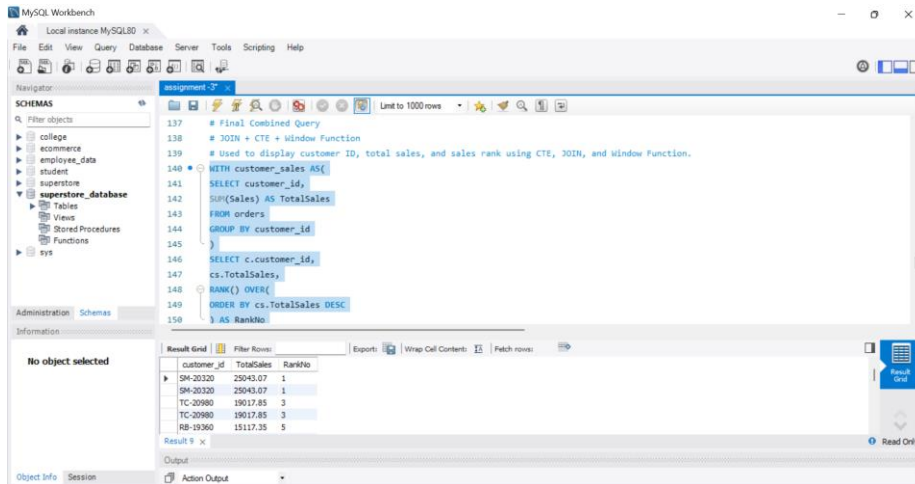
Step 3: Final Combined Query

Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

(Use **JOIN + CTE + Window Function** together)

WEEK -3 ASSIGNMENT



MySQL Workbench interface showing a query window with the following SQL code:

```
137 # Final Combined Query
138 # JOIN + CTE + Window Function
139 # Used to display customer ID, total sales, and sales rank using CTE, JOIN, and window Function.
140 WITH customer_sales AS(
141 SELECT customer_id,
142 SUM(Sales) AS TotalSales
143 FROM orders
144 GROUP BY customer_id
145 )
146 SELECT c.customer_id,
147 cs.TotalSales,
148 RANK() OVER(
149 ORDER BY cs.TotalSales DESC
150 ) AS RankNo
```

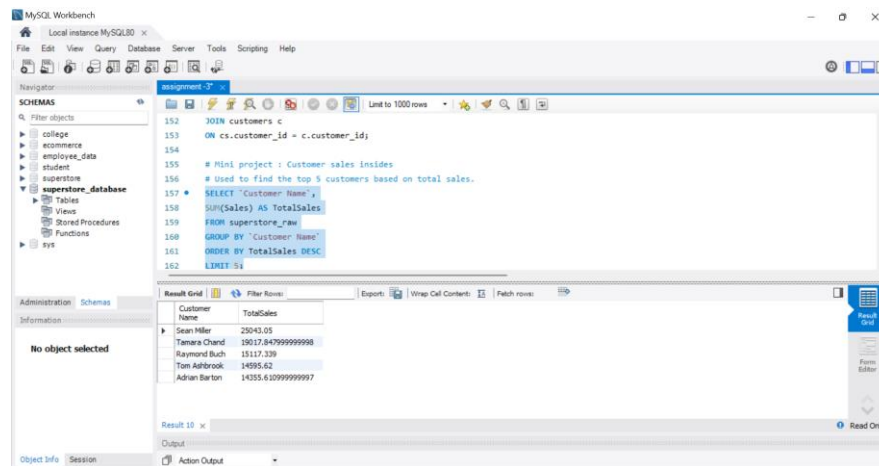
The result grid displays the following data:

customer_id	TotalSales	RankNo
SH-20320	25043.07	1
SH-20320	25043.07	1
TC-20980	19017.85	3
TC-20980	19017.85	3
RB-19360	15117.35	5

Mini Project: Customer Sales Insights

Answer the following using SQL:

1. Who are the top 5 customers?



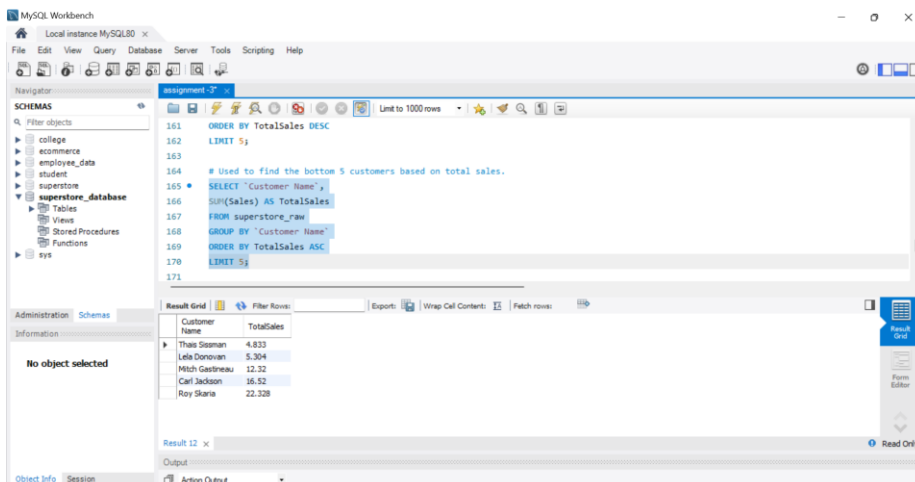
MySQL Workbench interface showing a query window with the following SQL code:

```
152 JOIN customers c
153 ON cs.customer_id = c.customer_id;
154
155 # Mini project : Customer sales insights
156 # Used to find the top 5 customers based on total sales.
157 SELECT "Customer Name",
158 SUM(Sales) AS TotalSales
159 FROM superstore_raw
160 GROUP BY "Customer Name"
161 ORDER BY TotalSales DESC
162 LIMIT 5;
```

The result grid displays the following data:

Customer Name	TotalSales
Sean Miller	25043.05
Tamara Chord	19017.849999999998
Raymond Buch	15117.339
Tom Ashbrook	14995.62
Adrian Barton	14355.610999999997

2. Who are the bottom 5 customers?



MySQL Workbench interface showing a query window with the following SQL code:

```
161 ORDER BY TotalSales DESC
162 LIMIT 5;
163
164 # Used to find the bottom 5 customers based on total sales.
165 SELECT "Customer Name",
166 SUM(Sales) AS TotalSales
167 FROM superstore_raw
168 GROUP BY "Customer Name"
169 ORDER BY TotalSales ASC
170 LIMIT 5;
171
```

The result grid displays the following data:

Customer Name	TotalSales
Thao Sisman	4.833
Lela Donohew	5.304
Nich Gathreau	12.32
Carl Jackson	16.52
Roy Skaria	22.328

WEEK -3 ASSIGNMENT

3. Which customers made only one order?

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
170 LIMIT 5;  
171  
172 # Used to identify customers who have placed only one order.  
173 * SELECT "Customer Name",  
174 FROM superstore_raw  
175 GROUP BY "Customer Name"  
176 HAVING COUNT(DISTINCT "Order ID") = 1;  
177  
178 # Used to find customers whose total sales are above the overall average sales.  
179 * SELECT "Customer Name",  
180 SUM(Sales) AS TotalSales
```

The Results tab shows a table with the following data:

Customer Name
Aronson, Nathan
Anthony O'Donnell
Carl Jackson
Jenna Caffrey
Jacobs, Rupert
Lila Donovan
Mitch Gashneau
Patricia Hirsaki
Rodgers Emerson

4. Which customers have above-average sales?

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
177  
178 # Used to find customers whose total sales are above the overall average sales.  
179 * SELECT "Customer Name",  
180 SUM(Sales) AS TotalSales,  
181 FROM superstore_raw  
182 GROUP BY "Customer Name"  
183 HAVING SUM(Sales) > (  
184 SELECT AVG(Sales)  
185 FROM superstore_raw  
186 )  
187
```

The Results tab shows a table with the following data:

Customer Name	TotalSales
Clare Gule	1148.7800000000002
Darien Van Huff	1119.4830000000002
Sean O'Donnell	2602.5750000000001
Brosina Hoffman	6255.3510000000001
Andrew Allen	1782.872
Irene Maddox	4930.4799999999999
Harold Pavilan	1990.314
Pete Kitz	8158.654

5. What is the highest order value per customer?

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
183 HAVING SUM(Sales) > (  
184 SELECT AVG(Sales)  
185 FROM superstore_raw  
186 )  
187  
188 # Used to display the highest order value recorded for each customer.  
189 * SELECT "Customer Name",  
190 MAX(Sales) AS HighestOrder  
191 FROM superstore_raw  
192 GROUP BY "Customer Name"  
193
```

The Results tab shows a table with the following data:

Customer Name	HighestOrder
Clare Gule	721.94
Darien Van Huff	721.875
Sean O'Donnell	957.5775
Brosina Hoffman	1706.184
Andrew Allen	499.87
Irene Maddox	1819.86
Harold Pavilan	1242.9
Pete Kitz	4305.552